

P1950R1

indirect_value: A Free-Store-Allocated Value Type For C++

Published Proposal, 2020-10-28

This version:

<https://wg21.link/P1950>

Issue Tracking:

[GitHub](#)

Authors:

[Jonathan B. Coe](#)

[Antony Peacock](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jbcoe/indirect_value/documentation/p1950.md

Table of Contents

- 1 Abstract**
- 2 Change history**
- 3 Introduction**
 - 3.1 Motivation
 - 3.1.1 Node-based containers
 - 3.1.2 Hot-cold splitting
 - 3.1.3 Pointer to Implementation - PImpl
 - 3.1.3.1 Issues with const-propagation

- 3.1.3.2 Issues with copies
- 3.1.3.3 An `indirect_value` implementation of the PImpl Idiom
- 3.2 Design decisions
 - 3.2.1 Should there be an empty state?
 - 3.2.2 Is there a small object optimization?
 - 3.2.3 How are allocators supported?
- 3.3 Prior Art
- 3.4 Completeness of T
- 3.5 Comparison with `unique_ptr<T>` and `polymorphic_value<T>`
- 3.6 Impact on the standard

4 Technical specifications

- 4.1 X.X Class template `default_copy` [default.copy]
- 4.2 X.Y Class template `indirect_value` [indirect_value]
 - 4.2.1 X.Y.1 Class template `indirect_value` general [indirect_value.general]
 - 4.2.2 X.Y.2 Class template `indirect_value` synopsis [indirect_value.synopsis]
 - 4.2.3 X.Y.3 Class template `indirect_value` constructors [indirect_value.ctor]
 - 4.2.4 X.Y.4 Class template `indirect_value` destructor [indirect_value.dtor]
 - 4.2.5 X.Y.5 Class template `indirect_value` assignment [indirect_value.assignment]
 - 4.2.6 X.Y.6 Class template `indirect_value` modifiers [indirect_value.modifiers]
 - 4.2.7 X.Y.7 Class template `indirect_value` observers [indirect_value.observers]
 - 4.2.8 X.Z.8 Class template `indirect_value` creation [indirect_value.creation]

5 Acknowledgements

6 References

§ 1. Abstract

Add a class template, `indirect_value`, to the C++ Standard Library to support free-store-allocated objects with value-like semantics.

§ 2. Change history

Changes in P1950r1

- Add design discussion.
- Add comparison with `unique_ptr` and `polymorphic_value`.
- Add node-based container and hot-cold splitting as further motivational examples.
- Add examples of similar classes already in use.

§ 3. Introduction

The class template, `indirect_value`, confers value-like semantics on a free-store-allocated object. An `indirect_value` may hold an object of a class `T`, copying the `indirect_value` will copy the object `T`. When a parent object contains a member of type `indirect_value<T>` and is accessed through a const access path, `const`ness will propagate from the parent object to the instance of `T` owned by the `indirect_value` member.

§ 3.1. Motivation

It may be desirable for a class member to be an incomplete type or for the storage used by a member object to be separate from the class itself. In both such cases, the member would traditionally be represented as pointer:

```
class MyClass {  
    AType data_member_;  
    AnotherType* indirect_data_member_;  
};
```

The author of such a class will need to implement special member functions as the compiler-generated special member functions will not function correctly (indirect data will not be copied, assigned to or deleted, only the pointer).

Special care will be needed when using the class in multithreaded environments. An instance of `MyClass` accessed through a const-access-path will not propagate the `const`-ness to the indirect data (only to the pointer itself) as pointers do not propagate `const` to their pointees.

The class template `indirect_value` is a drop-in replacement for pointer members in cases where non-polymorphic data referenced by the pointer member is logically part of the class. Use of `indirect_value` will ensure that the compiler-generated special member functions behave correctly and that `const` is propagated to indirect data.

§ 3.1.1. Node-based containers

A tree or linked list can be implemented as a node-based container. This gives stable references to data in the nodes at the cost of memory indirection when accessing data.

Nodes would typically contain data and pointer(s) to other nodes so that the list or tree can be navigated:

```
class ForwardListNode {
    int data_;
    ForwardListNode* next_;
};

class ForwardList {
    ForwardListNode* head_;
public:
    // ...
};
```

The special member functions of `ForwardList` would need to be user-implemented as the compiler-generated versions would not copy, move or delete the nodes correctly.

Care must be taken when implementing `const`-qualified member functions as `const` will not propagate down from the `ForwardList` to the `ForwardListNode`s nor to the data that the nodes contain.

Implementing the `ForwardList` and `ForwardListNode` with `indirect_value` corrects these issues:

```
class ForwardListNode {
    int data_;
    indirect_value<ForwardListNode> next_;
};

class ForwardList {
    indirect_value<ForwardListNode> head_;
public:
    // ...
};
```

Compiler-generated special member functions will behave correctly and do not need to be written manually. `const`-propagation will allow the compiler to ensure that data that is logically part of the

container cannot be modified through a `const` access path.

§ 3.1.2. Hot-cold splitting

When working with collections of data, CPU caches are often under-utilized when algorithms access certain data members much more frequently than others. This results in some cache lines becoming busier than others, prefetchers preemptively caching memory which is later evicted without being used by the CPU or memory bandwidth becoming a limiting factor in performance. In such cases segregating data structures in terms of hot and cold data can remove pressure on system resources.

```
class Element {
    SmallData frequently_accessed_data;
    LargeData infrequently_accessed_data;
};

vector<Element> elements;
auto active = find(elements.begin(),
                  elements.end(),
                  [] (const auto& e)
                  {
                      return e.frequently_accessed_data.active();
                  });
```

In such cases adding a level of indirection to the larger, less frequently-accessed data can relieve bandwidth pressure [C. Ericson]. Using `indirect_value` in this refactoring does not change guarantees around `const`ness. Compiler-generated special member functions will behave correctly and do not need to be manually implemented.

```
class Element {
    SmallData frequently_accessed_data;
    indirect_value<LargeData> infrequently_accessed_data;
};

vector<Element> elements;
auto active = find(elements.begin(),
                  elements.end(),
                  [] (const auto& e)
```

```

{
    return e.frequently_accessed_data.active();
};

```

§ 3.1.3. Pointer to Implementation - PImpl

In C++, when anything in a class definition changes, dependent classes require recompilation. As early as 1992 the Handle/Body idiom was suggested to break this dependency [J. Coplien]. From this pattern, the PImpl idiom was specialised [H. Sutter]. Despite its relative maturity, using the PImpl idiom requires careful thought about copying and `const`-propagation.

```

// Header file
class widget {
public:
    widget();
    ~widget();
private:
    class impl;
    std::unique_ptr<impl> pimpl_;
};

```

```

// Implementation file
class widget::impl {
    // :::
};

widget::widget() : pimpl_{ std::make_unique<impl>( /*...*/ ) } {}
// Destructor needs to be defined in a the same TU as <code data-opaque bs-
widget::~~widget() = default;

```

For convenience, the widget class will be referred to as the “visible class” and impl class the “**PImpl** class”. Note, semantically the **PImpl** class is the implementation details of the visible class.

§ 3.1.3.1. Issues with const-propagation

Using `std::unique_ptr` to store the implementation object introduces an issue - within a `const`-qualified member function, an instance of the visible class can mutate data inside the implementation

object. This is because `const`-qualification applies only to the `unique_ptr` value, and not the pointed-to-object.

The compiler is unable to make thread-compatibility assumptions for `const` objects when `const` does not propagate: `const` does not mean immutable in the face of pointer-like-member data.

The desired semantics of a PImpl-owner are value-like, like those of `std::optional` which has appropriate `const` and non-`const`-qualified overloads for `operator*` and `operator->`.

§ 3.1.3.2. Issues with copies

The copy-constructor and copy-assignment operator of `std::unique_ptr` are deleted. Users of a class with a `std::unique_ptr` member will be required to implement copy-construction and copy-assignment [S. Meyers].

§ 3.1.3.3. An `indirect_value` implementation of the PImpl Idiom

Using `indirect_value` to implement the PImpl idiom ensures that the PImpl object is `const` when accessed through a `const` access path and that compiler-generated special member functions will behave correctly and do not need to be manually implemented.

```
// Header file
class widget {
public:
    widget();
    widget(widget&& rhs) noexcept;
    widget(const widget& rhs);
    widget& operator=(widget&& rhs) noexcept;
    widget& operator=(const widget& rhs);
    ~widget();
private:
    class impl;
    std::indirect_value<impl>> pimpl;
};
```

```
// Implementation file
class widget::impl {
    // :::
};
```

```
// Special-member functions need to be defined in a the same TU as <code da
widget::widget(widget&& rhs) noexcept = default;
widget::widget(const widget& rhs) = default;
widget& widget::operator=(widget&& rhs) noexcept = default;
widget& widget::operator=(const widget& rhs) = default;
widget::~~widget() = default;
```

§ 3.2. Design decisions

§ 3.2.1. Should there be an empty state?

There is a design decision to be made about the default constructor of `indirect_value`. It could place `indirect_value` in an empty state with no owned object or it could default construct an owned object so that `indirect_value` is never empty.

If `indirect_value<T>` is never empty then `T` must be default constructible for `indirect_value<T>` to be a regular type. The default constructor of `indirect_value<T>` would then need to allocate memory, which cannot be controlled by the user-supplied copier or deleter.

If the default constructed state of `indirect_value` is non-empty then the moved from state should be similarly non-empty. This requires that `T` is noexcept move constructible.

Allowing an empty state for `indirect_value<T>` imposes preconditions on `operator*` and `operator->` (similar to those of the raw pointer it may be replacing) but removes requirements on `T` and the need to allocate memory.

A nullable `indirect_value<T>` could be mimicked with `std::optional<indirect_value<T>>` but this is verbose, does not solve the problem of `T` needing to be noexcept move-constructible and would require partial template specialization of `std::optional` to avoid overhead that would not be incurred by a nullable `indirect_value`.

As designed, `indirect_value` has an empty state where it owns no object. A default constructed `indirect_value` is empty and the empty state is the state of a moved-from object.

§ 3.2.2. Is there a small object optimization?

A small object optimization is permissible for `indirect_value`. A small buffer that is part of the `indirect_value` class can be used to store the owned object if it is small. We do not require that

`indirect_value` is the same size as a raw pointer; implementers are free, but not required, to add a small buffer optimization as a quality of implementation improvement.

It would be possible for the size of the small buffer to be specified as a template argument. Such a design would be inconsistent with the broader Standard Library design: `std::string`, `std::any`, `std::function` all permit a small object optimization but do not let users specify the size of the buffer.

§ 3.2.3. How are allocators supported?

It may be desirable for users to control the way memory is managed by an instance of a class. Classes such as `std::vector` and `std::string` allow an allocator to be specified as template argument to control memory management.

Like `std::unique_ptr`, `indirect_value` allows a deleter to be provided as a template argument so that disposal of the owned object can be customized. `indirect_value` also allows copying to be customized by specifying a copier, another template argument.

The in-place constructor of `indirect_value<T>` uses `new` to create an instance of `T` with the supplied constructor argument. This constructor is disabled where the copier and deleter are not `default_copy` and `default_delete`.

Memory management for `indirect_value` can be fully controlled by specifying a suitable copier and deleter and using the pointer constructor `indirect_value(T* t, C c, D d)`.

§ 3.3. Prior Art

There have been previous proposal for deep-copying smart pointers that proposed copy semantics [W. Brown]. `cloned_ptr` was proposed in [J. Coe], however under guidance of LEWG this was renamed to `polymorphic_value`. With this change in name came the addition of const propagation.

This paper is not unique in these ideas. GitHub code search finds 602k lines of code referencing "Pimpl" and 99 C++ repositories claiming to provide generic implementations of Pimpl. Additionally other authors have addressed this topic [A. Upadyshev]. Some generic implementations of note in the wild are:

Project	Link	Deep Copying	Const Propagation
Boost Pimpl	https://github.com/sean-/boost-pimpl	Yes	Yes

Project	Link	Deep Copying	Const Propagation
pimpl	https://github.com/JonChesterfield/pimpl	Yes	No
impl_ptr	https://github.com/sth/impl_ptr	Yes	Yes
Simpl	https://github.com/oliora/samples/blob/master/spimpl.h	Yes	Yes
smart_pimpl	https://github.com/SBreezyx/smart_pimpl	No	No
pimpl_on_stack	https://github.com/kuznetsss/pimpl_on_stack/	Yes	Yes
deep_const_ptr	https://github.com/torbjoernk/deep_const_ptr	No	Yes

Divergence on issues such as deep copying and const propagation along with subtleties in implementation mean that a single standardized solution would be of broad benefit to the community of C++ users.

§ 3.4. Completeness of T

Smart pointer types in the Standard Library expect that some of the members can be instantiated with incomplete types [H.Hinnant]. Similarly, this is the case for `indirect_value`, the table outlines the expected behaviour for incomplete pointer types:

Method	Description	Incomplete/Complete
<code>indirect_value()</code>	Default constructor	Incomplete
<code>indirect_value(const indirect_value&)</code>	Copy-constructor	Complete
<code>indirect_value(indirect_value&&)</code>	Move-constructor	Incomplete
<code>~indirect_value()</code>	Destructor	Complete
<code>indirect_value& indirect_value::operator=(const indirect_value&)</code>	Copy-assignment	Complete
<code>indirect_value& indirect_value::operator=(indirect_value&&)</code>	Move-assignment	Complete
<code>T& operator*()</code>	Indirection-operator	Incomplete
<code>const T& operator*() const</code>	Indirection-operator	Incomplete
<code>T* operator->() noexcept</code>	Member-of-pointer-operator	Incomplete

Method	Description	Incomplete/Complete
<code>const T* operator->() const noexcept</code>	Member-of-pointer-operator	Incomplete
<code>explicit operator bool() const noexcept</code>	Bool-operator	Incomplete
<code>void swap(indirect_value& p) noexcept</code>	Swap	Incomplete

§ 3.5. Comparison with `unique_ptr<T>` and `polymorphic_value<T>`

The class template `indirect_value<T>` is in many respects similar to `unique_ptr<T>` and the proposed `polymorphic_value<T>` [J. Coe]. The table below highlights the similarities and differences:

Behaviour	<code>indirect_value</code>	<code>polymorphic_value</code>	<code>unique_ptr</code>
Default constructed state	Empty	Empty	Empty
Copyable	Yes	Yes	No
Const-propagating	Yes	Yes	No
Polymorphic	No	Yes	Yes
Customizable memory access	Yes	No	Yes

§ 3.6. Impact on the standard

This proposal is a pure library extension. It requires additions to be made to the standard library header `<memory>`.

§ 4. Technical specifications

§ 4.1. X.X Class template `default_copy` [default.copy]

```
namespace std {
    template <class T>
    struct default_copy {
        T* operator()(const T& t) const;
    };
} // namespace std
```

The class template `default_copy` serves as the default copier for the class template `indirect_value`. The template parameter `T` of `default_copy` may be an incomplete type.

```
T* operator()(const T& t) const;
```

- Returns: `new T(t);`

§ 4.2. X.Y Class template `indirect_value` [`indirect_value`]

§ 4.2.1. X.Y.1 Class template `indirect_value` general [`indirect_value.general`]

An `indirect_value` is an object that owns another object and manages that other object through a pointer. More precisely, an indirect value is an object `v` that stores a pointer to a second object `p` and will dispose of `p` when `v` is itself destroyed (e.g., when leaving block scope (9.7)). In this context, `v` is said to own `p`.

An `indirect_value` object is empty if it does not own a pointer.

Copying a non-empty `indirect_value` will copy the owned object so that the copied `indirect_value` will have its own unique copy of the owned object.

Copying from an empty `indirect_value` produces another empty `indirect_value`.

Copying and disposal of the owned object can be customised by supplying a copier and deleter.

The template parameter `T` of `indirect_value` must be a non-union class type.

The template parameter `T` of `indirect_value` may be an incomplete type.

[Note: Implementations are encouraged to avoid the use of dynamic memory for ownership of small objects.]

§ 4.2.2. X.Y.2 Class template `indirect_value` synopsis [`indirect_value.synopsis`]

```
template <class T, class C = std::default_copy<T>, class D = std::default_deleter<T>>
class indirect_value {
public:
    using value_type = T;

    // Constructors
    constexpr indirect_value() noexcept;
```

```

    explicit indirect_value(T* p, C c=C{}, D d=D{});

    indirect_value(const indirect_value& p);
    indirect_value(indirect_value&& p) noexcept;

    template <class ...Ts>
    indirect_value(std::in_place_t, Ts&&... ts); // See below

    // Destructor
    ~indirect_value();

    // Assignment
    indirect_value& operator=(const indirect_value& p);
    indirect_value& operator=(indirect_value&& p) noexcept;

    // Modifiers
    void swap(indirect_value<T>& p) noexcept;

    // Observers
    T& operator*();
    T* operator->();
    const T& operator*() const;
    const T* operator->() const;
    explicit operator bool() const noexcept;
};

// indirect_value creation
template <class T, class ...Ts> indirect_value<T>
make_indirect_value(Ts&& ...ts); // See below

// indirect_value specialized algorithms
template<class T>
void swap(indirect_value<T>& p, indirect_value<T>& u) noexcept;

} // end namespace std

```

§ 4.2.3. X.Y.3 Class template `indirect_value` constructors [indirect_value.ctor]

```
constexpr indirect_value() noexcept;
```

- Remarks: The method shall work with incomplete pointer type for `T`.

- Effects: Constructs an empty `indirect_value`.
 - Postconditions: `bool(*this) == false`.

```
explicit indirect_value(T* p, C c=C{}, D d=D{});
```

- Effects: Creates an `indirect_value` object that owns the pointer `p`. If `p` is non-null then the copier and deleter of the `indirect_value` constructed are moved from `c` and `d`.
- Requires: `C` and `D` satisfy the requirements of CopyConstructible. If `p` is non-null then the expression `c(*p)` returns an object of type `T*`. The expression `d(p)` is well-formed, has well-defined behaviour, and does not throw exceptions.
- Postconditions: `bool(*this) == bool(p)`.
- Remarks: A custom copier and deleter are said to be ‘present’ in a `indirect_value` initialised with this constructor.

```
indirect_value(const indirect_value& p);
```

- Constraints: `is_copy_constructible_v<T>`
- Effects: Creates a `indirect_value` object that owns a copy of the object managed by `p`. The copy is created by the copier in `p`. If `p` has a custom copier and deleter then the custom copier and deleter of the `indirect_value` constructed are copied from those in `p`.
- Throws: Any exception thrown by the copier or `bad_alloc` if required storage cannot be obtained.
- Postconditions: `bool(*this) == bool(p)`.

```
indirect_value(indirect_value&& p) noexcept;
```

- Effects: Move-constructs an `indirect_value` instance from `p`. If `p` has a custom copier and deleter then the copier and deleter of the `indirect_value` constructed are the same as those in `p`.
- Postconditions: `*this` contains the old value of `p`. `p` is empty.
- Remarks: The method shall work with incomplete pointer type for `T`.

```
indirect_value(std::in_place_t, Ts&& ...ts);
```

- Effects: Constructs an `indirect_value` which owns an object of type `T` direct-non-list-initialized with `std::forward<Ts>(ts)...`
- Throws: Any exception thrown by the selected constructor of `T` or `bad_alloc` if required storage cannot be obtained.

- Requires `is_same_v<C, default_copy> && is_same_v<D, default_delete>`.

§ 4.2.4. X.Y.4 Class template `indirect_value` destructor [`indirect_value.dtor`]

```
~indirect_value();
```

- Effects: If `get() == nullptr` there are no effects. If a custom deleter `d` is present then `d(p)` is called and the copier and deleter are destroyed. Otherwise the destructor of the managed object is called.

§ 4.2.5. X.Y.5 Class template `indirect_value` assignment [`indirect_value.assignment`]

```
indirect_value& operator=(const indirect_value& p);
```

- Constraints: `is_copy_assignable<T>`
- Effects: `*this` owns a copy of the resource managed by `p`. If `p` has a custom copier and deleter then the copy is created by the copier in `p`, and the copier and deleter of `*this` are copied from those in `p`. Otherwise, the resource managed by `*this` is initialised by the copy constructor of the resource managed by `p`.
- Throws: Any exception thrown by the copier or `bad_alloc` if required storage cannot be obtained.
- Returns: `*this`.
- Postconditions: `bool(*this) == bool(p)`.

```
indirect_value& operator=(indirect_value&& p) noexcept;
```

- Effects: Ownership of the resource managed by `p` is transferred to this. If `p` has a custom copier and deleter then the copier and deleter of `*this` is the same as those in `p`.
- Returns: `*this`.
- Postconditions: `*this` contains the old value of `p`. `p` is empty.

§ 4.2.6. X.Y.6 Class template `indirect_value` modifiers [`indirect_value.modifiers`]

```
void swap(indirect_value& p) noexcept;
```

- Effects: Exchanges the contents of `p` and `*this`.

- Remarks: The method shall work with incomplete pointer type for `T`.

§ 4.2.7. X.Y.7 Class template `indirect_value` observers [`indirect_value.observers`]

```
T& operator*();
const T& operator*() const;
```

- Requires: `bool(*this)`.
- Returns: A reference to the owned object.

```
T* operator->() noexcept;
const T* operator->() const noexcept;
```

- Requires: `bool(*this)`.
- Returns: A pointer to the owned object.
- Remarks: The method shall work with incomplete pointer type for `T`.

```
explicit operator bool() const noexcept;
```

- Returns: false if the `indirect_value` is empty, otherwise true.
- Remarks: The method shall work with incomplete pointer type for `T`.

§ 4.2.8. X.Z.8 Class template `indirect_value` creation [`indirect_value.creation`]

```
template <class T, class U=T, class ...Ts> indirect_value<T>
    make_indirect_value(Ts&& ...ts);
```

- Constraints: `is_constructible_v<U, Ts...>` is true.
- Expects: `U` meets the `Cpp17CopyConstructible` requirements.
- Returns: A `indirect_value<T>` owning an object of type direct-non-list-initialized with `std::forward<Ts>(ts)...`.
- Requires `is_same_v<C, default_copy> && is_same_v<D, default_delete>`.

§ 5. Acknowledgements

The authors would like to thank Thomas Russell, and Andrew Bennieston for useful discussions on the topic and the BSI panel for on-going support.

§ 6. References

[J. Coe] [p0201r3: A polymorphic value-type for C++](#)

[J. Coplien] Advanced C++ Programming Styles and Idioms (Addison-Wesley), James O. Coplien, 1992

[C. Ericson] [Memory Optimization, Christer Ericson, Games Developers Conference, 2003](#)

[A. Upadyshev] [PIMPL, Rule of Zero and Scott Meyers, Andrey Upadyshev, 2015](#)

[H. Hinnant] ["Incomplete types and shared_ptr / unique_ptr", Howard Hinnant, 2011](#)

[H. Sutter] ["Pimpls - Beauty Marks You Can Depend On", Herb Sutter, 1998](#)

[Impl] [Reference implementation: indirect_value, J.B.Coe](#)

[S. Meyers] Effective Modern C++, Item 22: When using the Pimpl Idiom, define special member functions in the implementation file, Scott Meyers, 2014

[W. Brown] [n3339: A Preliminary Proposal for a Deep-Copying Smart Pointer, Walter E. Brown, 2012](#)